📚 Smart Research Assistant (RAG-Based Knowledge System)

Overview

This project is a **Retrieval-Augmented Generation (RAG)** based AI assistant that helps users interact with large document collections through natural language queries. It is designed especially for interns to understand how modern AI systems combine **LLMs, vector databases, and evaluation frameworks** into a real-world application.

The system allows users to upload documents (PDFs, text files, web content) and ask questions. The assistant retrieves relevant information and generates accurate, context-aware responses backed by source data.

Use Case

Researchers, or employees often need to quickly extract insights from large volumes of documents such as:

- Research papers
- Company policies
- Technical documentation

Instead of manually reading everything, users can simply ask questions like:

“What is the leave policy?”

“Summarize this research paper”

The system responds with:

- A precise answer
- Supporting document context
- Evaluation metrics for reliability

Core Concept

The system follows a structured pipeline:

1. User submits a query
2. Query is converted into embeddings
3. Relevant document chunks are retrieved from a vector database
4. Retrieved context is passed to an LLM
5. LLM generates a response grounded in retrieved data

This ensures answers are **fact-based and context-aware**, unlike generic LLM outputs.

Technology Stack

LangChain

Used to orchestrate the entire pipeline:

- Document loading
- Text chunking
- Retrieval chains
- Prompt management

FAISS

- Performs fast similarity search
- Lightweight and efficient for local setups
- Ideal for prototyping

ChromaDB

- Persistent vector storage
- Supports scalable applications
- Useful for production-like environments

OpenAI API or any other alternative

- Used for generating high-quality responses
- Provides advanced language models (GPT family)

Hugging Face

- Provides open-source embedding models
- Enables cost-effective or offline setups

RAGs

Used to evaluate system performance:

- Faithfulness (is the answer grounded in context?)
- Relevance (does it answer the question?)
- Context precision

Streamlit

- Builds a simple and interactive web interface
- Used for document upload and chat interaction

Gradio

- Alternative UI framework
- Useful for quick demos and experimentation

System Workflow

1. Data Ingestion

- Upload documents (PDF, TXT, etc.)
- Split into smaller chunks
- Generate embeddings using OpenAI or Hugging Face models

2. Storage

- Store embeddings in FAISS or ChromaDB
- Enable fast retrieval during queries

3. Query Processing

- Convert user query into embedding
- Retrieve top relevant document chunks

4. Answer Generation

- Combine query + retrieved context
- Generate answer using OpenAI LLM

5. Evaluation

- Use RAGAs to measure:
    - Answer correctness
    - Context alignment
    - Overall quality

Key Features

Core Features

- Document upload and processing
- Context-aware question answering
- Source-backed responses
- Vector-based search

Advanced Features

- Switch between FAISS and ChromaDB
- Multiple embedding model options
- Multi-turn conversational memory
- Highlight relevant document sections

Evaluation Features

- Automated scoring using RAGs
- Display quality metrics in UI
- Helps improve system reliability

Example Scenario

Step 1: Upload HR policy documents

Step 2: Ask:

“What is the maternity leave policy?”

System Output:

- Clear, concise answer
- Extracted policy text as reference
- Evaluation score indicating answer reliability

Learning Outcomes

By building this project, interns will understand:

- How LLMs integrate with external knowledge
- Working of vector databases and embeddings
- Prompt engineering fundamentals
- Retrieval-based AI system design
- Importance of evaluation in AI systems

Future Enhancement(extra)

- Voice-based interaction
- Multi-language support
- Cloud deployment (AWS/GCP)
- Domain-specific assistants (legal, healthcare, finance)
- User authentication for enterprise use

Conclusion

This project demonstrates how modern AI systems move beyond standalone language models by combining **retrieval, reasoning, and evaluation**. It provides a strong foundation for building scalable, reliable, and production-ready AI applications.